Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 6

Дисциплина МДК 02.02
Клиентская архитектура MVVM и OpenAPI

Вариант № 2
АИС «Электронная медицинская карта»

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ РАБОТЫ.....3

1.1 Слои MVVM.....3

1.2 Контракт OpenAPI.....4

1.3 Реализация и обработка ошибок.....4

2 ПРОВЕРКА РЕЗУЛЬТАТОВ.....6

ЗАКЛЮЧЕНИЕ.....7

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....8

ПРИЛОЖЕНИЕ А.....9

А.1 Листинг PatientViewModel.....9

А.2 Листинг MedicalApiClient.....11

А.3 Листинг PatientWindow.....12

А.4 Листинг Program.....13

1 ВЫПОЛНЕНИЕ РАБОТЫ

Цель работы — разделить клиент на View, ViewModel и Model; описать два взаимосвязанных эндпоинта OpenAPI 3.0.3 и реализовать C#-клиент. Работа выполнена по варианту 2 нового сборника [2].

В примерах используются вымышленные пациенты. Оформление выполнено по требованиям института [1].

1.1 Слои MVVM

WPF Binding связывает свойства ViewModel с элементами окна, а INotifyPropertyChanged уведомляет интерфейс об изменениях [3]. ViewModel зависит от IMedicalApiClient, поэтому проверяется без запуска окна WPF.

Результат показан на рисунке 1.

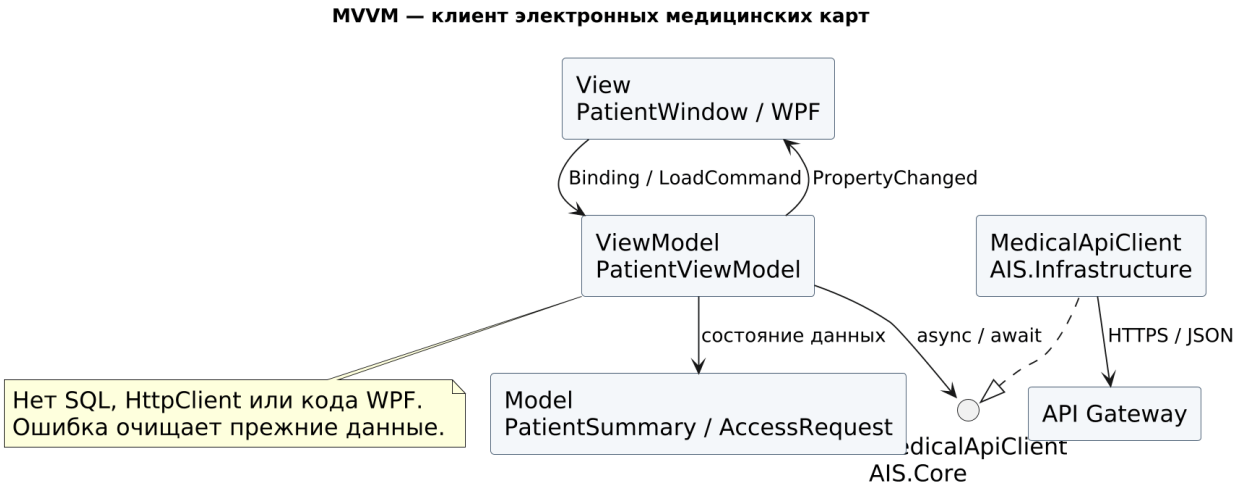


Рисунок 1 — Связи View, ViewModel, Model и сетевого адаптера

Источник: схема разработана для учебного проекта.

Сведения представлены в таблице 1.

Таблица 1 — Распределение MVVM

Слой	Класс или проект	Ответственность
View	AIS.UI / PatientWindow	Binding, кнопка команды и отображение
ViewModel	AIS.Presentation / PatientViewModel	Состояние, LoadCommand, IsBusy, ошибки
Model	AIS.Core / PatientSummary	Данные краткой карты
Сетевой адаптер	AIS.Infrastructure / MedicalApiClient	HTTP, Bearer, JSON, коды ошибок
Точка сборки	AIS.Desktop / Program	Создание клиента и связывание зависимостей

Источник: составлено по учебному заданию [2].

1.2 Контракт OpenAPI

Контракт содержит GET /api/v1/patients/{id}/summary и POST /api/v1/access/check. Повторно используются схемы PatientSummary, AccessRequest, AccessReply и ApiError. Bearer задаётся как security scheme; роль в запросе отсутствует [4].

```
openapi: 3.0.3
info:
  title: EMR Integration Gateway
  version: 1.0.0
security:
  - BearerAuth: []
paths:
  /api/v1/patients/{id}/summary:
    get:
      operationId: getPatientSummary
      # параметры и responses — в полном openapi.yaml
  /api/v1/access/check:
    post:
      operationId: checkAccess
```

Полный валидный YAML приложен в contracts/openapi.yaml; приведённый фрагмент показывает расположение операций. Ошибки описаны по HTTP-кодам, а модели вынесены в components/schemas. Фрагмент не заменяет полный контракт.

1.3 Реализация и обработка ошибок

LoadCommand проверяет UUID, очищает прежние сведения, включает IsBusy и вызывает GetSummaryAsync. На успехе обновляются имя, дата

рождения и статус. На 403, сетевой ошибке или неверном JSON выводится сообщение, а IsBusy сбрасывается в finally.

```
dotnet run --project src/AIS.Gateway -- --demo
# Во втором терминале Windows:
dotnet run --project src/AIS.Desktop
```

После старта API токен учебного врача копируется из консоли в поле подключения WPF-клиента. HTTP допускается только для локального учебного адреса; внешний адрес клиента должен использовать HTTPS.

2 ПРОВЕРКА РЕЗУЛЬТАТОВ

OpenAPI прошёл проверку спецификации. C#-тесты выполняют HTTP-запросы, десериализацию, LoadAsync, PropertyChanged, сброс IsBusy и очистку данных после ошибки.

Результат показан на рисунке 2.

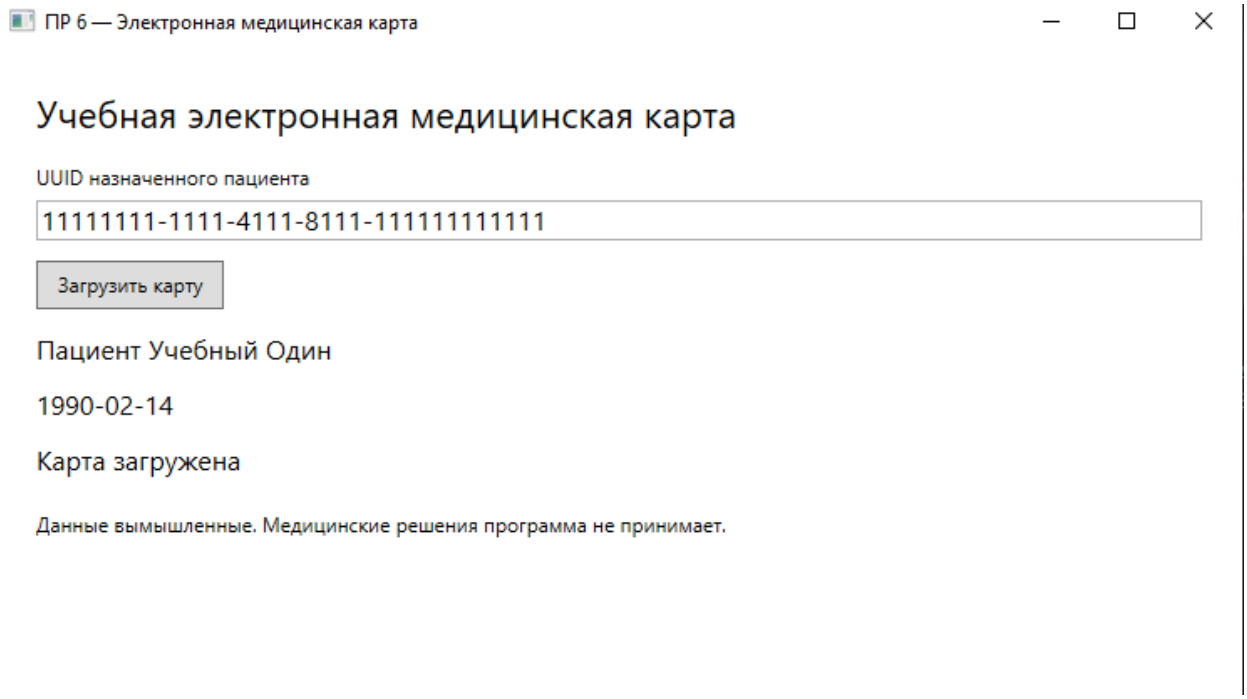


Рисунок 2 — Загрузка карты пациента в WPF-клиенте

Источник: снимок окна WPF-клиента при запуске на Windows.

На рисунке 2 показано окно WPF-клиента, запущенного на Windows. После команды «Загрузить карту» отображаются имя пациента, дата рождения и статус «Карта загружена». Полные листинги View, ViewModel и сетевого клиента находятся в приложении А.

Исходники и материалы актуальной публикации: https://github.com/rob4ik72-del/MDK0202-EMR

ЗАКЛЮЧЕНИЕ

Создан WPF-клиент с MVVM и отдельным сетевым адаптером. Бизнес-состояние не зависит от элементов окна или SQL.

Контракт OpenAPI 3.0.3 валиден и соответствует маршрутам шлюза. Положительные и отрицательные сценарии ViewModel проверены отдельно от View.

Репозиторий проекта: https://github.com/rob4ik72-del/MDK0202-EMR

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. СПбИЭУ. Положение о требованиях по подготовке и оформлению письменных работ и приложение М: локальный нормативный акт, предоставленный в материалах обучения. [1].

2. МДК 02.02. Детализированный сборник практических работ ПР1–ПР9: электронный учебный материал; вариант 2 «Электронная медицинская карта». [2].

3. Microsoft Learn. WPF data binding overview. URL: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/data/ (дата обращения: 05.11.2026) [3].

4. OpenAPI Initiative. OpenAPI Specification 3.0.3. URL: https://spec.openapis.org/oas/v3.0.3 (дата обращения: 05.11.2026) [4].

ПРИЛОЖЕНИЕ А

А.1 Листинг PatientViewModel

src/AIS.Presentation/PatientViewModel.cs

```
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows.Input;
using Ais.Core;

namespace Ais.Presentation;
public sealed class PatientViewModel : INotifyPropertyChanged
{
   readonly IMedicalApiClient api;
   string patientId = SampleData.PatientId.ToString(), status = "Готово к загрузке", name = "", birth = "";
   bool busy;
   public PatientViewModel(IMedicalApiClient client)
   {
      api = client;
      LoadCommand = new AsyncCommand(LoadAsync, () => !IsBusy);
   }

   public string PatientId
   {
      get => patientId;
      set
      {
         patientId = value;
         Changed();
      }
   }

   public string Status
   {
      get => status;
      private set
      {
         status = value;
         Changed();
      }
   }

   public string PatientName
   {
      get => name;
      private set
      {
         name = value;
         Changed();
      }
   }

   public string BirthDate
   {
      get => birth;
```

```
        private set
        {
            birth = value;
            Changed();
        }
    }

    public bool IsBusy
    {
        get => busy;
        private set
        {
            busy = value;
            Changed();
            LoadCommand.Refresh();
        }
    }

    public AsyncCommand LoadCommand { get; }

    public event PropertyChangedEventHandler? PropertyChanged;
    void Changed([CallerMemberName] string? property = null) => PropertyChanged?.Invoke(this,
new(property));
    public async Task LoadAsync()
    {
        if (IsBusy)
            return;
        PatientName = "";
        BirthDate = "";
        if (!Guid.TryParse(PatientId, out var id) || id == Guid.Empty)
        {
            Status = "Введите непустой UUID пациента";
            return;
        }

        IsBusy = true;
        Status = "Загрузка";
        try
        {
            var result = await api.GetSummaryAsync(id);
            PatientName = result.FullName;
            BirthDate = result.BirthDate;
            Status = "Карта загружена";
        }
        catch (HttpRequestException e)
        {
            Status = e.Message;
        }
        catch (InvalidDataException e)
        {
            Status = e.Message;
        }
        catch (OperationCanceledException)
        {
            Status = "Загрузка отменена";
        }
        finally
        {
```

```
            IsBusy = false;
        }
    }
}

public sealed class AsyncCommand(Func<Task> action, Func<bool> canExecute) : ICommand
{
    bool running;
    public event EventHandler? CanExecuteChanged;
    public bool CanExecute(object? parameter) => !running && canExecute();
    public async void Execute(object? parameter)
    {
        if (!CanExecute(parameter))
            return;
        running = true;
        Refresh();
        try
        {
            await action();
        }
        finally
        {
            running = false;
            Refresh();
        }
    }

    public void Refresh() => CanExecuteChanged?.Invoke(this, EventArgs.Empty);
}
```

A.2 Листинг MedicalApiClient

```
src/AIS.Infrastructure/MedicalApiClient.cs
using System.Net;
using System.Net.Http.Headers;
using System.Net.Http.Json;
using Ais.Core;

namespace Ais.Infrastructure;
public sealed class MedicalApiClient : IMedicalApiClient
{
    readonly HttpClient http;
    public MedicalApiClient(HttpClient client, string token, bool allowLoopbackDemo = false)
    {
        http = client;
        if (http.BaseAddress == null || (http.BaseAddress.Scheme != "https" && !(allowLoopbackDemo &&
http.BaseAddress.IsLoopback)))
            throw new ArgumentException("Нужен HTTPS; HTTP разрешён только локальному учебному
серверу");
        if (string.IsNullOrWhiteSpace(token))
            throw new ArgumentException("Не указан токен");
        http.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);
    }

    public async Task<PatientSummary> GetSummaryAsync(Guid id, CancellationToken ct = default)
    {
```

```
        using var response = await http.GetAsync($"api/v1/patients/{id}/summary", ct);
        await Ensure(response, ct);
        return await Read<PatientSummary>(response, ct);
    }

    public async Task<AccessReply> CheckAccessAsync(AccessRequest request, CancellationToken ct = default)
    {
        using var response = await http.PostAsJsonAsync("api/v1/access/check", request, ct);
        await Ensure(response, ct);
        return await Read<AccessReply>(response, ct);
    }

    static async Task<T> Read<T>(HttpResponseMessage response, CancellationToken ct)
    {
        try
        {
            return await response.Content.ReadFromJsonAsync<T>(cancellationToken: ct) ?? throw new
InvalidDataException("Пустой ответ API");
        }
        catch (System.Text.Json.JsonException e)
        {
            throw new InvalidDataException("Некорректный JSON ответа", e);
        }
    }

    static async Task Ensure(HttpResponseMessage response, CancellationToken ct)
    {
        if (response.IsSuccessStatusCode)
            return;
        string message = response.StatusCode switch
        {
            HttpStatusCode.Unauthorized => "Требуется действующий токен",
            HttpStatusCode.Forbidden => "Нет доступа к этой карте",
            HttpStatusCode.BadRequest => "Ошибка параметров запроса",
            _ => "Шлюз не выполнил запрос"
        };
        await response.Content.LoadIntoBufferAsync();
        ct.ThrowIfCancellationRequested();
        throw new HttpRequestException(message, null, response.StatusCode);
    }
}
```

A.3 Листинг PatientWindow

```
src/AIS.UI/PatientWindow.cs
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using Ais.Presentation;

namespace Ais.UI;
public sealed class PatientWindow : Window
{
    public PatientWindow(PatientViewModel viewModel)
    {
        Title = "ПР 6 — Электронная медицинская карта";
```

```
        Width = 760;
        Height = 430;
        DataContext = viewModel;
        var panel = new StackPanel
        {
            Margin = new Thickness(24)
        };
        panel.Children.Add(new TextBlock { Text = "Учебная электронная медицинская карта", FontSize = 22,
Margin = new Thickness(0, 0, 0, 16) });
        panel.Children.Add(new TextBlock { Text = "UUID назначенного пациента" });
        var id = new TextBox
        {
            FontSize = 16,
            Margin = new Thickness(0, 6, 0, 12)
        };
        id.SetBinding(TextBox.TextProperty, new Binding("PatientId") { UpdateSourceTrigger =
UpdateSourceTrigger.PropertyChanged });
        panel.Children.Add(id);
        var load = new Button
        {
            Content = "Загрузить карту",
            Padding = new Thickness(12, 6, 12, 6),
            HorizontalAlignment = HorizontalAlignment.Left
        };
        load.SetBinding(Button.CommandProperty, new Binding("LoadCommand"));
        panel.Children.Add(load);
        foreach (var property in new[]
        {
            "PatientName",
            "BirthDate",
            "Status"
        }


        )
        {
            var text = new TextBlock
            {
                FontSize = 16,
                Margin = new Thickness(0, 12, 0, 0),
                TextWrapping = TextWrapping.Wrap
            };
            text.SetBinding(TextBlock.TextProperty, new Binding(property));
            panel.Children.Add(text);
        }


        panel.Children.Add(new TextBlock { Text = "Данные вымышленные. Медицинские решения
программа не принимает.", Margin = new Thickness(0, 20, 0, 0), TextWrapping = TextWrapping.Wrap });
        Content = panel;
    }
}
```

A.4 Листинг Program

```
src/AIS.Desktop/Program.cs
using System.Net.Http;
using System.Windows;
using System.Windows.Controls;
```

```
using Ais.Infrastructure;
using Ais.Presentation;
using Ais.UI;

namespace Ais.Desktop;
public static class Program
{
   [STAThread]
   public static void Main()
   {
      var app = new Application();
      var connection = new Window
      {
         Title = "Подключение к учебному API ЭМК",
         Width = 690,
         Height = 330
      };
      var panel = new StackPanel
      {
         Margin = new Thickness(24)
      };
      panel.Children.Add(new TextBlock { Text = "Сначала запустите AIS.Gateway с параметром --demo.",
TextWrapping = TextWrapping.Wrap });
      panel.Children.Add(new TextBlock { Text = "Адрес API", Margin = new Thickness(0, 12, 0, 0) });
      var url = new TextBox
      {
         Text = "http://127.0.0.1:5274"
      };
      panel.Children.Add(url);
      panel.Children.Add(new TextBlock { Text = "Doctor token из консоли API", Margin = new Thickness(0,
12, 0, 0) });
      var token = new PasswordBox();
      panel.Children.Add(token);
      var button = new Button
      {
         Content = "Открыть клиент",
         Margin = new Thickness(0, 16, 0, 0),
         Padding = new Thickness(10)
      };
      panel.Children.Add(button);
      button.Click += (_, _) =>
      {
         try
         {
            if (!Uri.TryCreate(url.Text, UriKind.Absolute, out var address))
               throw new ArgumentException("Некорректный адрес API");
            var http = new HttpClient
            {
               BaseAddress = address,
               Timeout = TimeSpan.FromSeconds(15)
            };
            var client = new MedicalApiClient(http, token.Password, address.IsLoopback);
            var window = new PatientWindow(new PatientViewModel(client));
            window.Closed += (_, _) => http.Dispose();
            app.MainWindow = window;
            window.Show();
            connection.Close();
         }
         catch (ArgumentException e)
```

```
            {
                MessageBox.Show(e.Message, "Ошибка подключения");
            }
        };
        connection.Content = panel;
        app.Run(connection);
    }
}
```